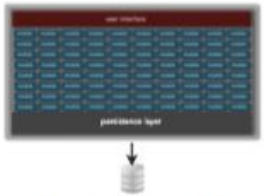
Hochschule Konstanz
Fakultät Informatik

Software Architectures

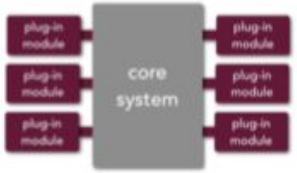
13 - Spark

Prof. Dr. Marko Boger

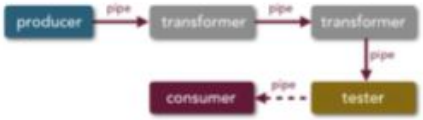
Architecture Patterns



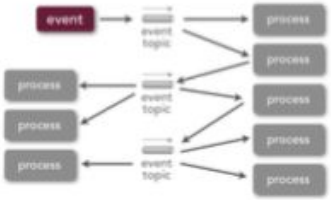
layered



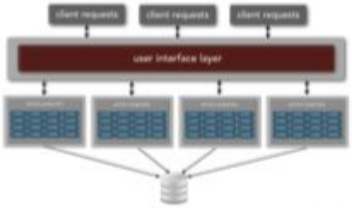
microkernel



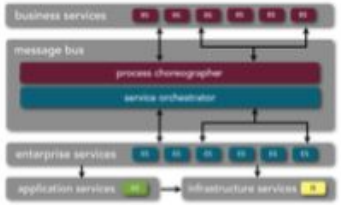
pipeline



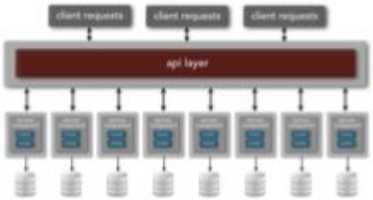
event-driven



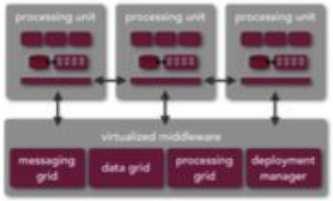
service-based



service-oriented



microservices



space-based

Architecture for Uber or Lyft

https://www.youtube.com/watch?v=R_agd5qZ26Y

Spark

Spark is a cluster-computing framework.

It is written in Scala and is open source.

It was first introduced in 2009.

It has APIs for Java, Scala, Python, and R.

Functional concepts like Immutability, Closures, Higher Order Functions etc. are very important in Spark, so it feels most natural in Scala.

Current version is 4.0.0.

What are Cluster-Computing Frameworks?

Cluster is a term meaning independent computers combined into a unified system through software and networking. Clusters are typically used for High Availability for greater reliability or High Performance Computing to provide greater computational power than a single computer can provide.

Popular frameworks include Hadoop, Storm, Flink and Spark.

Hadoop was one of the first open-source frameworks for cluster computing, released in 2006. It is modeled after a paper by Google about their BigTable, first released 2005.

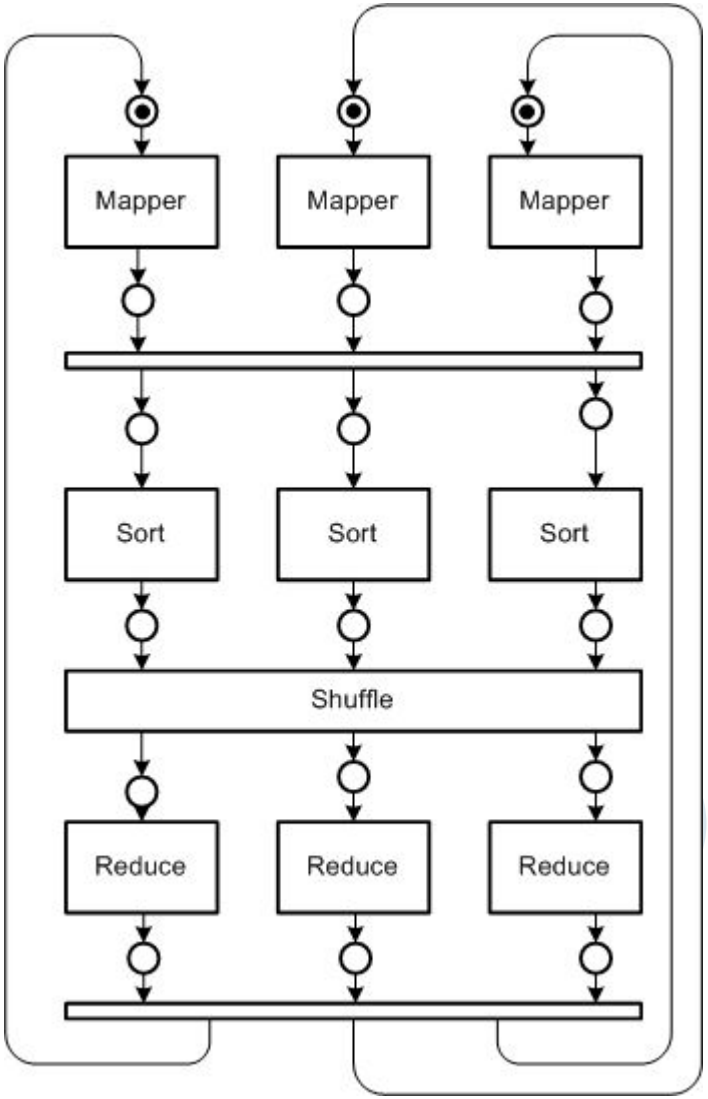
Hadoop and BigTable are distributed, wide-column databases, but they can also very effectively run algorithms on this data with a mechanism called Map/Reduce.

Map/Reduce

Google's Framework for distributed data processing

Idea:

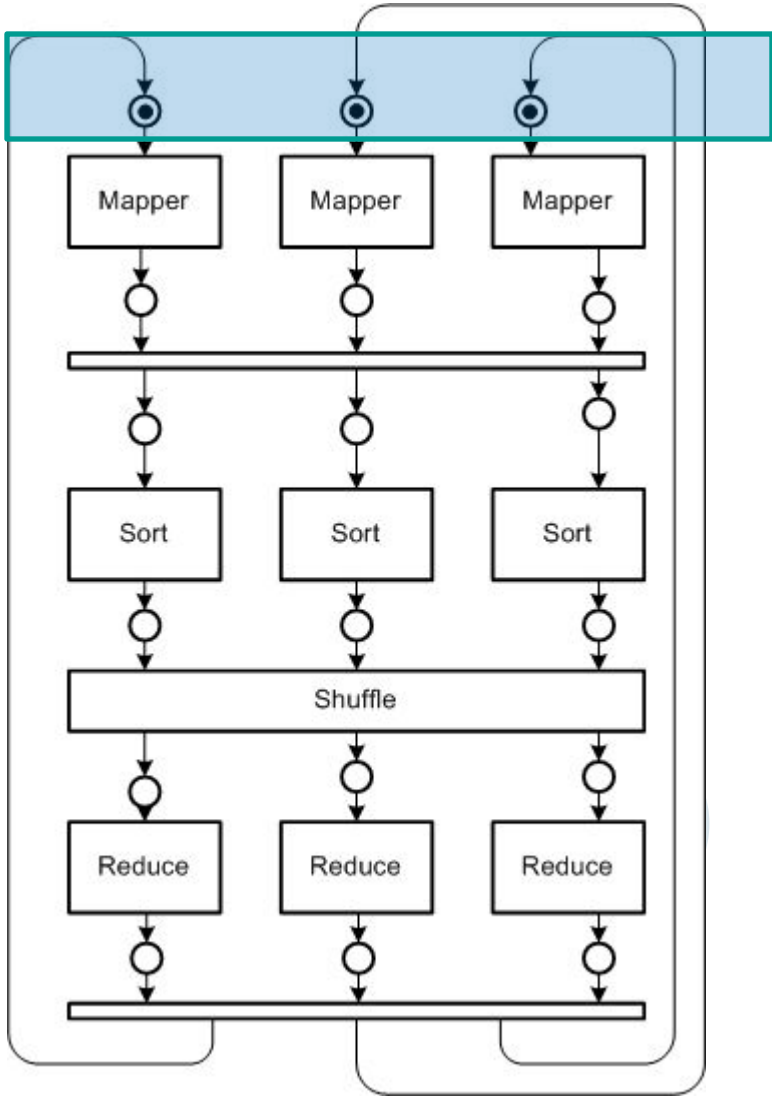
- Shared-Nothing
- Data processing distributed over worker nodes
- Mapping defines dependencies between data records
- Reducer defines aggregation operation



Map/Reduce – Blog Analysis

```
function map(document) {
        emit(document.author, 1);
}
```

```
function reduce(key, values) {
        emit(key, values.length);
}
```



Map/Reduce – Blog Analysis

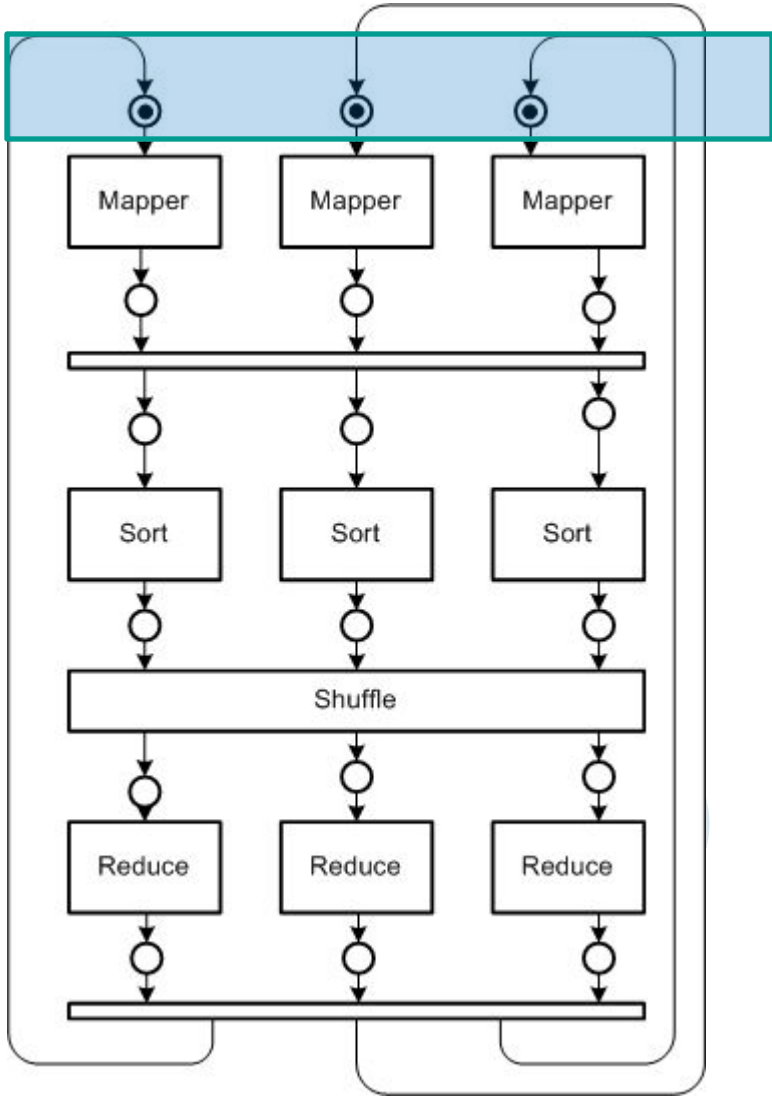
Task: Find number of blog-entries per author

```
function map(document) {
        emit(document.author, 1);
}
```

```
function reduce(key, values) {
        emit(key, values.length);
}
```

Input data:

{ „id“ = „123“, „text“ = „…“, author = „john doe“ }
{ „id“ = „124“, „text“ = „…“, author = „jane doe“ }
{ „id“ = „125“, „text“ = „…“, author = „jane doe“ }
{ „id“ = „126“, „text“ = „…“, author = „john doe“ }
{ „id“ = „127“, „text“ = „…“, author = „max doe“ }



Map/Reduce – Blog Analysis

Task: Find number of blog-entries per author

```
function map(document) {
        emit(document.author, 1);
}
```

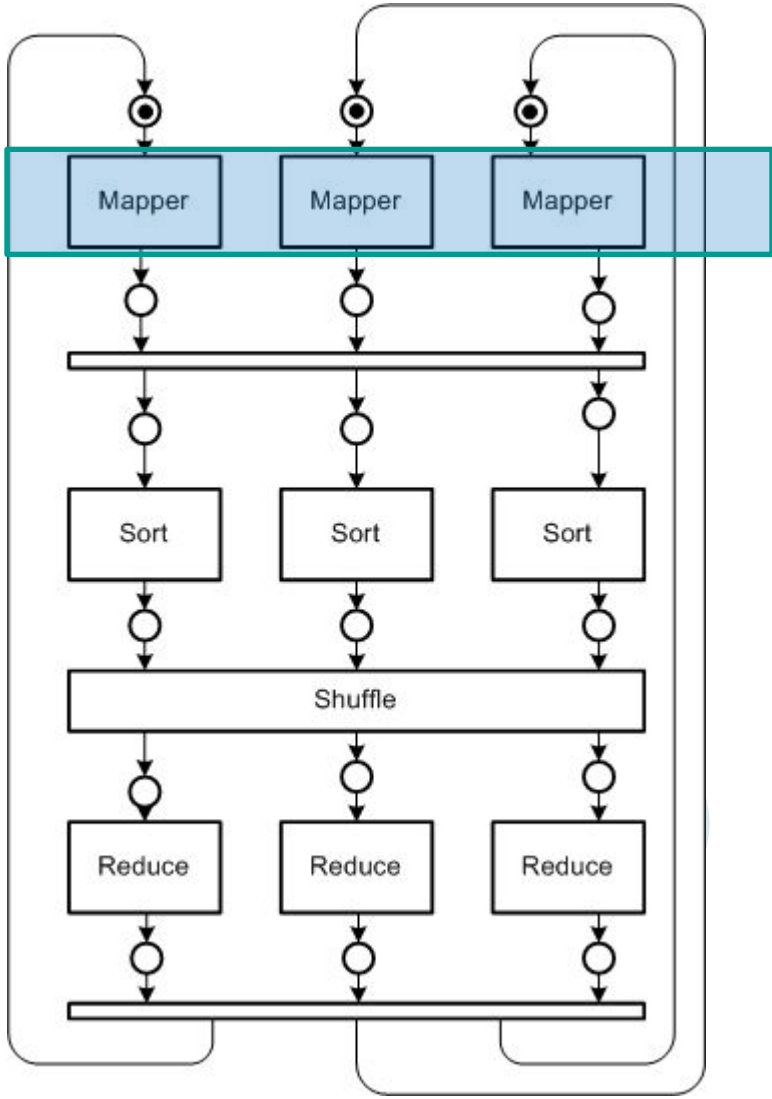
```
function reduce(key, values) {
        emit(key, values.length);
}
```

Input data:

{
{
{
{

Mapped data:

„Key“ = „john doe“ , „Value“ = „1“
„Key“ = „jane doe“ , „Value“ = „1“
„Key“ = „jane doe“ , „Value“ = „1“
„Key“ = „john doe“ , „Value“ = „1“
„Key“ = „max doe“ , „Value“ = „1“



Map/Reduce – Blog Analysis

Task: Find number of blog-entries per author

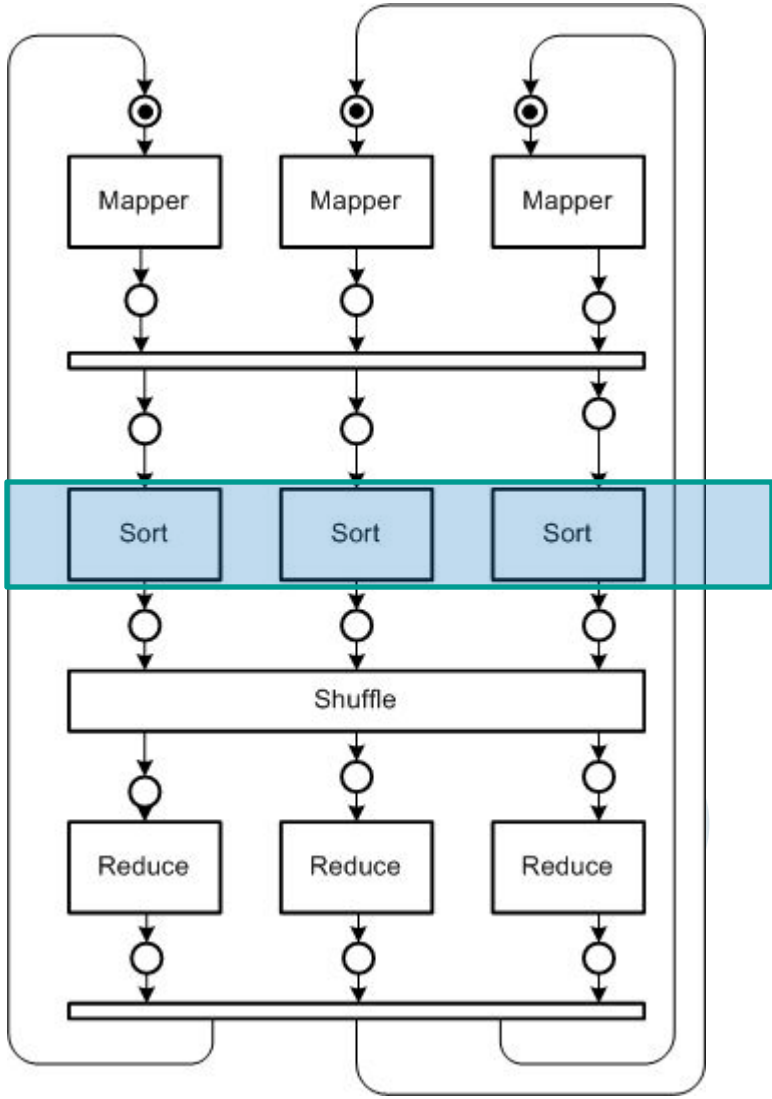
```
function map(document) {
        emit(document.author, 1);
}
```

```
function reduce(key, values) {
        emit(key, values.length);
}
```

Input data:

Sorted data:

„Key" = „jane doe", „Value" = „1"
„Key" = „jane doe", „Value" = „1"
„Key" = „john doe", „Value" = „1"
„Key" = „john doe", „Value" = „1"
„Key" = „max doe", „Value" = „1"

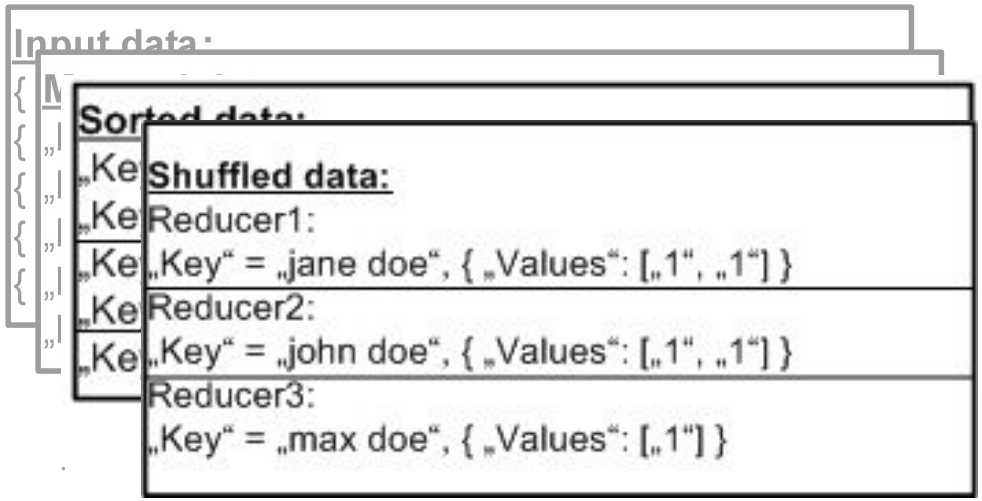


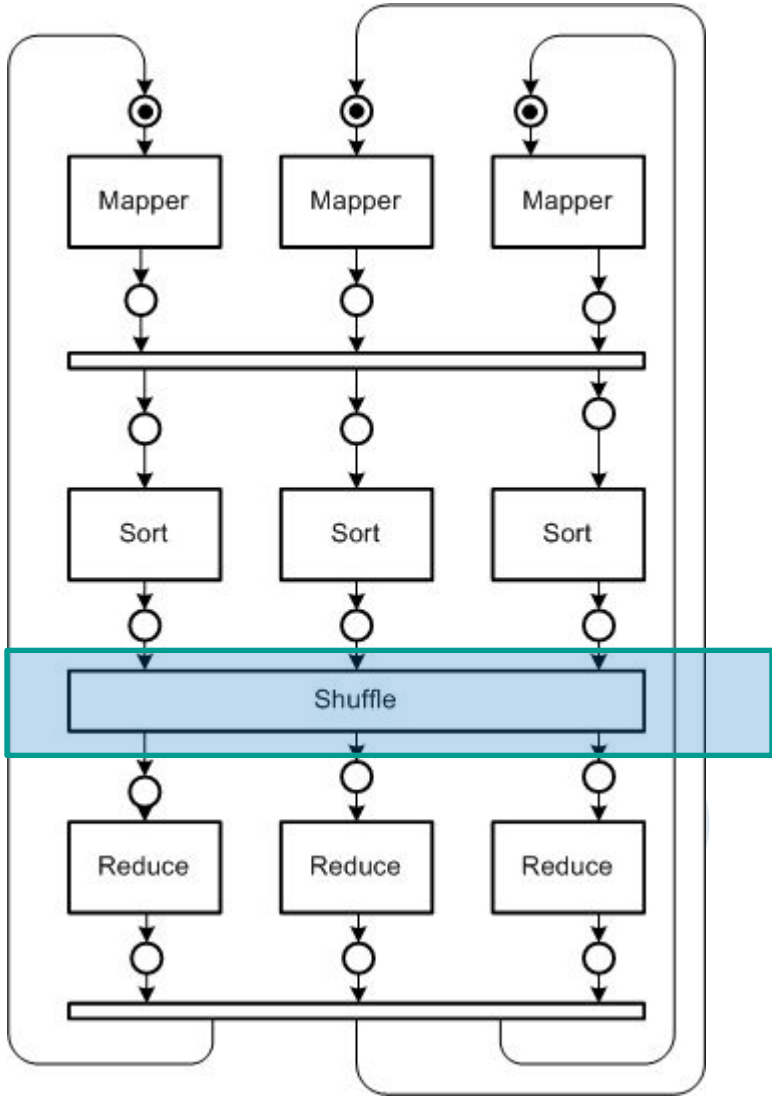
Map/Reduce – Blog Analysis

Task: Find number of blog-entries per author

```
function map(document) {
        emit(document.author, 1);
}
```

```
function reduce(key, values) {
        emit(key, values.length);
}
```



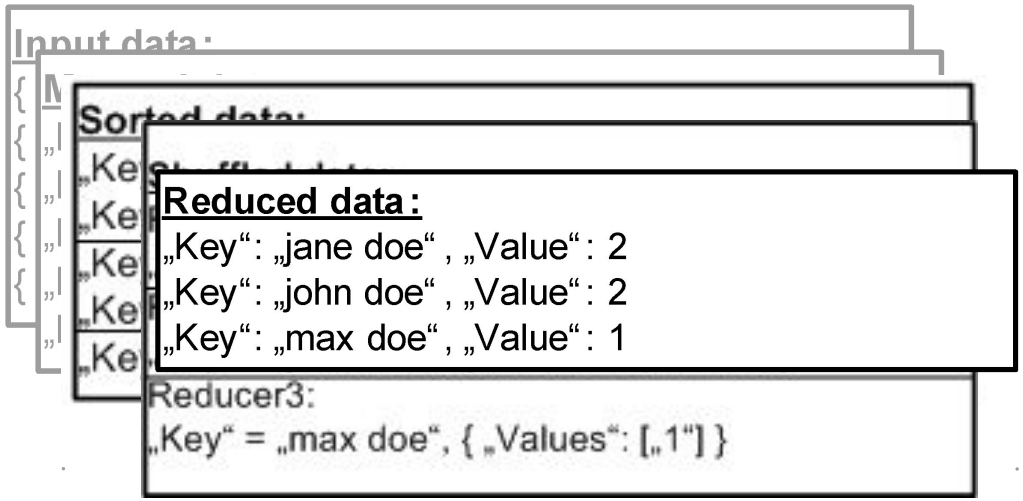


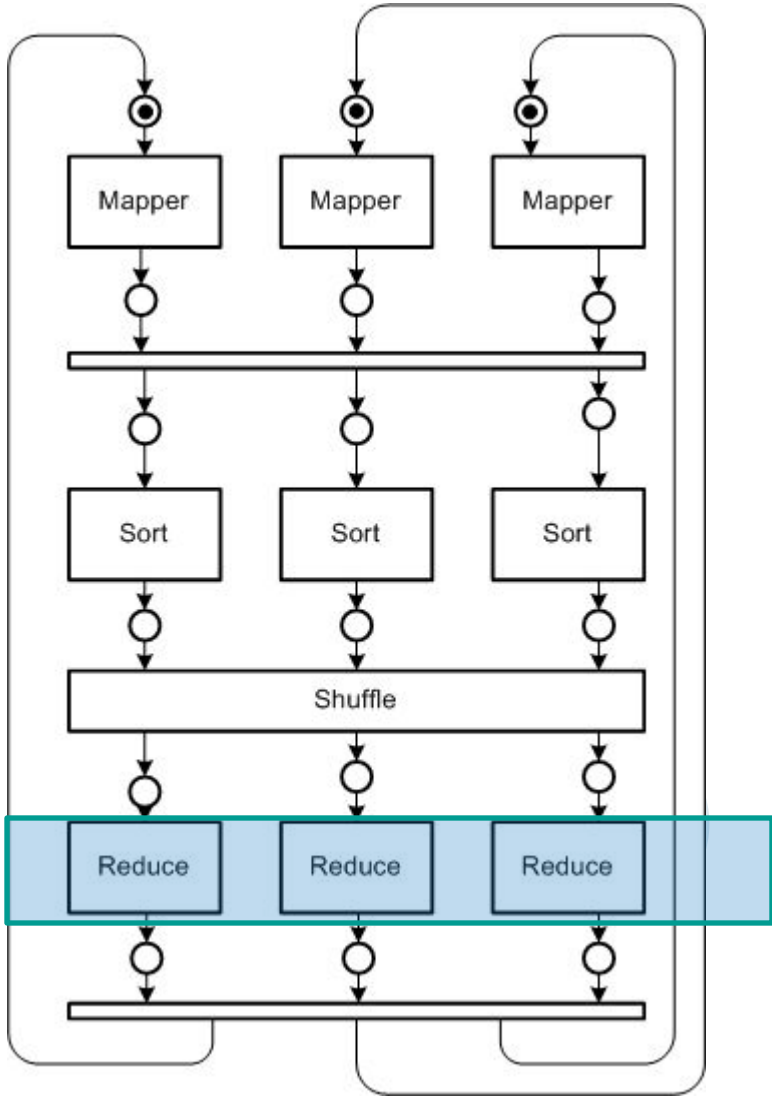
Map/Reduce – Blog Analysis

Task: Find number of blog-entries per author

```
function map(document) {
        emit(document.author, 1);
}
```

```
function reduce(key, values) {
        emit(key, values.length);
}
```





Typical Cluster operations

A cluster can efficiently calculate operations that involve large sets of data and aggregations or transformations that are associative in nature.

For example sorting, counting, filtering etc.

The “Hello World” example of BigData is WordCount. So lets take a look.

Plain Scala Word Count

```
val list = List("In Reactive Systems we take a closer look at technologies for BigData","Spark is a BigData technology","BigData")
val words = list.flatMap(line => line.split(" "))
val keyData = words.map(word => (word,1))
val groupedData = keyData.groupBy(_._1)
val result = groupedData.mapValues(list=>{
 list.map(_._2).sum
})
result.foreach(println)
```

or

```
list.flatMap(line => line.split(" ")).map(word => (word,1)).groupBy(_._1).mapValues(list=>{
 list.map(_._2).sum}).foreach(println)
```

Output:

(for,1)
(BigData,3)
(is,1)
(In,1)
(look,1)
(closer,1)
(a,2)
(technologies,1)
(technology,1)
(Reactive,1)
(at,1)
(take,1)
(Systems,1)
(Spark,1)
(we,1)

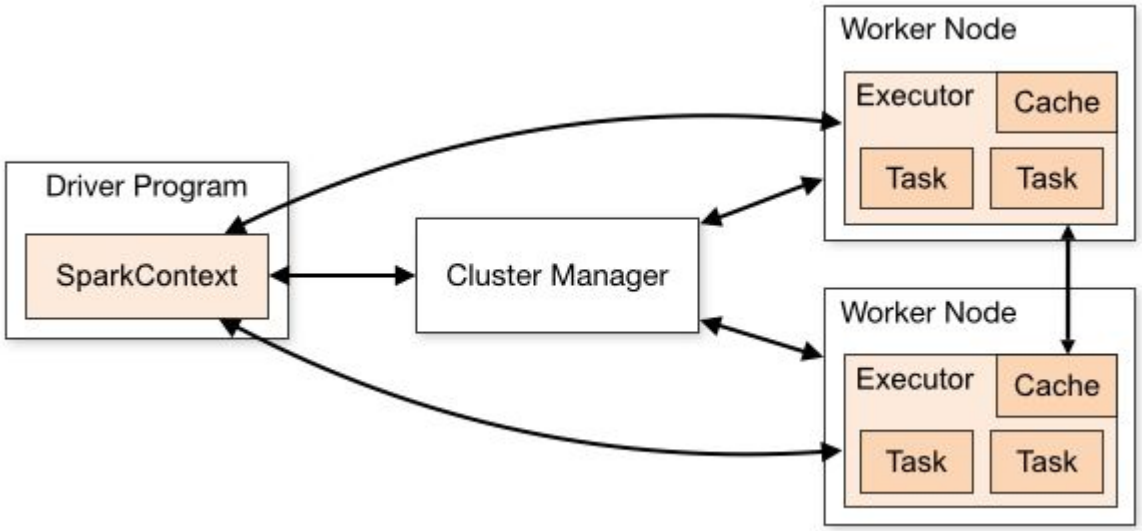
Spark Word Count

```
val textFile = sc.textFile("hdfs://...")
val counts = textFile.flatMap(line => line.split(" "))
                 .map(word => (word, 1))
                 .reduceByKey(_ + _)
 counts.saveAsTextFile("hdfs://...")
```

Spark Architecture

Spark has these main parts

- The Driver Program runs the show, much like a conductor.
- The Cluster Manager distributes tasks to Worker Nodes
  - Cluster Manager could be either Spark, YARN, Mesos or Hadoop
- The Worker Nodes run the Executor
- The Executor run the Tasks
- The Tasks are chunks of the original Driver Program that can be parallelized



The Main Data Structure: RDDs or Dataset

In version 2, the main data structure was called RDD, starting with version 3, the main data structure now is Dataset.

The RDD or Dataset is much like a regular Collection, but it has Distribution built in. RDD stands for Resilient Distributed Datasets.

RDDs and Datasets are created by the driver program. Within the driver program, the RDD/Dataset looks and feels like a collection.

We will continue with Datasets, but the term RDD will show up in videos, documentation or examples.

Transformations on Datasets

Transformations are functions that can be called on Datasets, that can run on an Executor. They include

- map
- flatmap
- filter
- distinct
- sample
- union, intersection, subtract, cartesian

Transformations can be composed. But they are executed lazily. Execution is enforced by Actions.

Actions

Only when an action is called, the transformations required to execute the action are calculated. They are similar to the reduce step im Map/Reduce. Actions enforce a shuffle, so they are usually expensive.

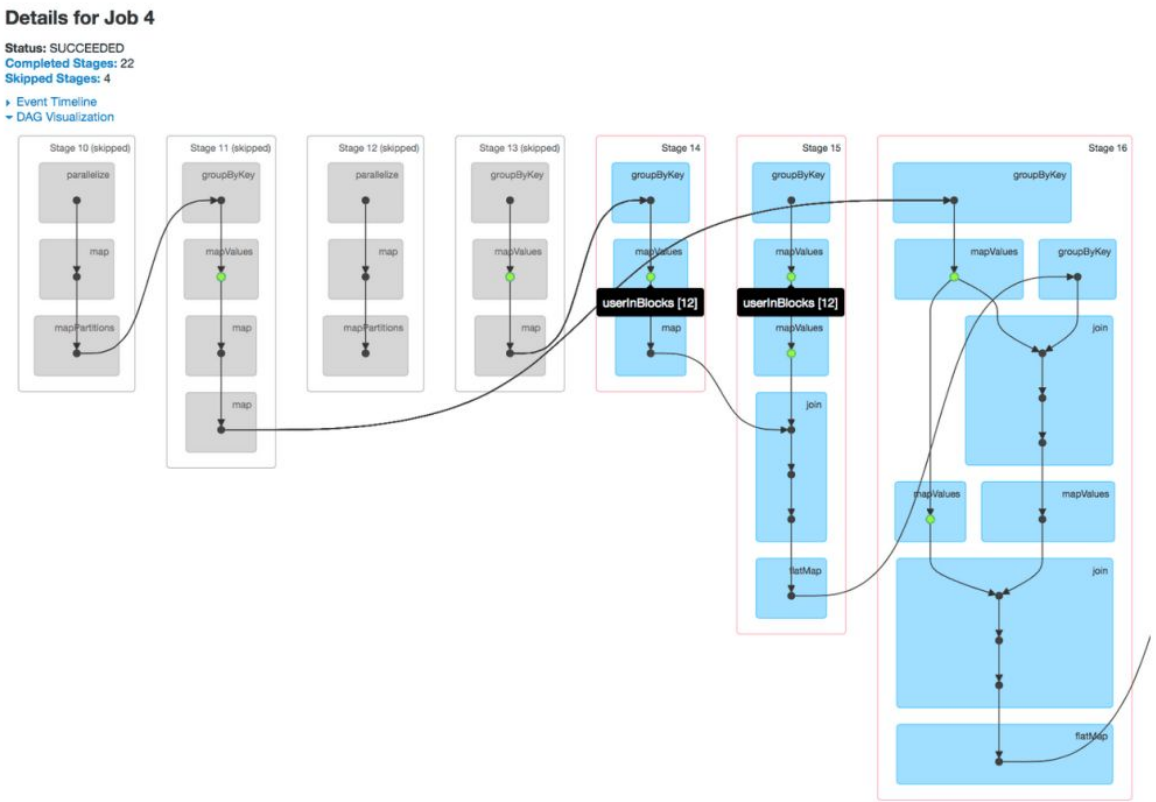
Actions include

- take
- count
- countByValue
- top
- reduce

Spark is more than Map/Reduce

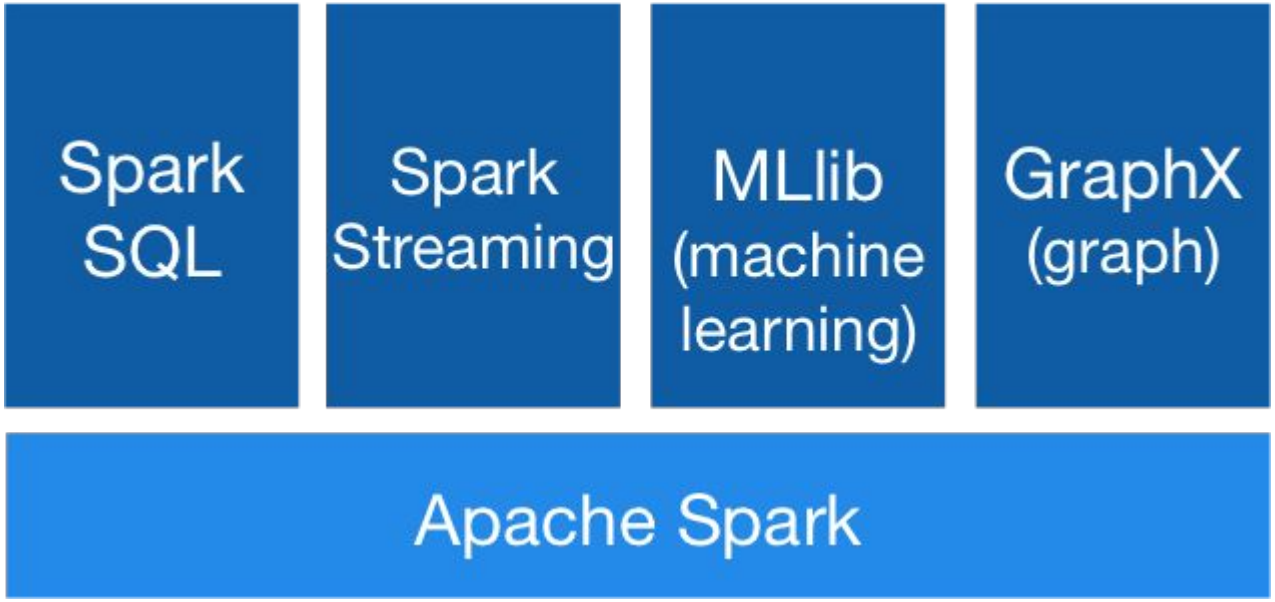
Spark can do map/reduce. But it is not limited to that.

Transformations and Actions can be combined to a complex graph.

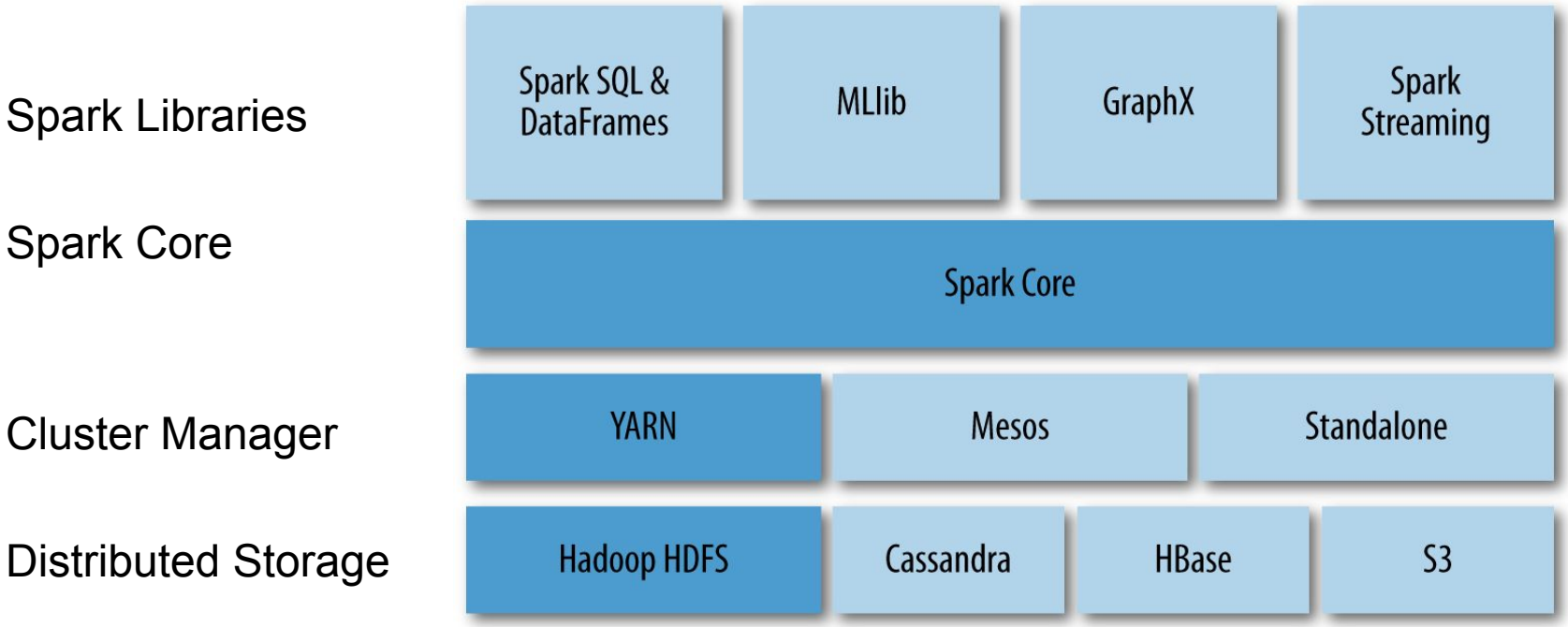


Spark Libraries

Next to the core, there are additional libraries for specific domains



Spark on top of Distributed File Systems



Installing Spark

Spark is a runtime environment, not just a library. So we need to download it from https://spark.apache.org/downloads.html

Current version is 3.5.0. Download it with Hadoop 3.3. Then unpack in a folder and cd into that folder. You can run examples with

```
./bin/run-example SparkPi 10
```

The library is installed from sbt. It is available for Scala 2.12 and Java 8 or 11

```
libraryDependencies += "org.apache.spark" % "spark-core_2.12" % "3.0.1"
libraryDependencies += "org.apache.spark" % "spark-streaming_2.12" % "3.0.1"
libraryDependencies += "org.apache.spark" % "spark-streaming-kafka-0-10_2.12" % "3.0.1"
libraryDependencies += "org.apache.spark" % "spark-sql_2.12" % "3.0.1"
```

Spark in the shell

Spark can be run in the shell, much like the Scala REPL.

```
./bin/spark-shell
```

```
Spark context Web UI available at http://10.0.1.11:4040
Spark context available as 'sc' (master = local[*], app id = local-1607940770366).
Spark session available as 'spark'.

scala> val textFile = spark.read.textFile("README.md")
textFile: org.apache.spark.sql.Dataset[String] = [value: string]

scala> textFile.count()
res1: Long = 108

scala> textFile.first
res2: String = # Apache Spark

scala> textFile.count
res3: Long = 108

scala> val linesWithSpark = textFile.filter(line => line.contains("Spark"))
linesWithSpark: org.apache.spark.sql.Dataset[String] = [value: string]

scala> linesWithSpark.count
res4: Long = 19
```

Spark in IDE

Any Spark App requires a SparkSession

```
import org.apache.spark.sql.SparkSession

object SimpleApp {
 def main(args: Array[String]) {
  val spark = SparkSession.builder.appName("Simple Application").config("spark.master", "local").getOrCreate()
  ...
  spark.stop()
 }
}
```

and should stop it again

First Example

```
import org.apache.spark.sql.SparkSession

object SimpleApp {
 def main(args: Array[String]) {
   val spark = SparkSession.builder.appName("Simple Application").config("spark.master", "local").getOrCreate()

   val logData = spark.read.textFile("./README.md").cache()
   val numAs = logData.filter(line => line.contains("a")).count()
   val numBs = logData.filter(line => line.contains("b")).count()
   println(s"Lines with a: $numAs, Lines with b: $numBs")

   spark.stop()
 }
}
```

Lines with a: 8, Lines with b: 5

Approximation of Pi, with and without Spark

```scala
import scala.math.random

object LocalPi {
 def main(args: Array[String]): Unit = {

  var count = 0
  for (i <- 1 to 100000) {
   val x = random * 2 - 1
   val y = random * 2 - 1
   if (x*x + y*y <= 1) count += 1
  }
  println(s"Pi is roughly ${4 * count / 100000.0}")
 }
}
```

```scala
import scala.math.random
import org.apache.spark.sql.SparkSession

object SparkPi {
 def main(args: Array[String]): Unit = {
  val spark = SparkSession.builder.appName("Spark Pi").config("spark.master",
"local").getOrCreate()

  val slices = if (args.length > 0) args(0).toInt else 2
  val n = math.min(100000L * slices, Int.MaxValue).toInt // avoid overflow
  val count = spark.sparkContext.parallelize(1 until n, slices).map { i =>
   val x = random * 2 - 1
   val y = random * 2 - 1
   if (x*x + y*y <= 1) 1 else 0
  }.reduce(_ + _)
  println(s"Pi is roughly ${4.0 * count / (n - 1)}")

  spark.stop()
 }
}
```

Spark Streaming

Spark Streaming can directly connect to Kafka.

The documentation is here:
https://spark.apache.org/docs/latest/streaming-kafka-0-10-integration.html

But it did not work as documented. I got it to compile, and to run, but not to do anything useful. Here is my code:

```
import org.apache.spark.sql.SparkSession
import org.apache.spark.streaming.kafka010._
import org.apache.spark.streaming.{Seconds, StreamingContext}
import org.apache.spark.streaming.kafka010.LocationStrategies.PreferConsistent
import org.apache.spark.streaming.kafka010.ConsumerStrategies.Subscribe

import collection.JavaConverters.asJavaCollection
import scala.collection.JavaConverters._
import org.apache.kafka.common.TopicPartition
```

```
object SparkStream {
 def main(args:Array[String]):Unit = {
   val spark = SparkSession.builder.appName("Spark Streaming").config("spark.master", "local").getOrCreate()

   val scalaKafkaParams = Map[String, Object](
   "bootstrap.servers" -> "localhost:9092",
   "key.deserializer" -> classOf[StringDeserializer],
   "value.deserializer" -> classOf[StringDeserializer],
   "group.id" -> "use_a_separate_group_id_for_each_stream",
   "auto.offset.reset" -> "latest",
   "enable.auto.commit" -> (false: java.lang.Boolean)
   )
   val kafkaParams=scalaKafkaParams.asJava
   val sc = new StreamingContext(spark.sparkContext, Seconds(1))

   val topics = asJavaCollection(Array("topicA", "topicB"))
   val offset = Map[TopicPartition, java.lang.Long]((new TopicPartition("partition",1),1L)).asJava

   val stream = KafkaUtils.createDirectStream[String, String](sc, PreferConsistent, Subscribe[String, String](topics, kafkaParams,
offset))

   stream.foreachRDD(rdd => println( rdd.toString()))

   spark.stop()
 }
}
```

Task Spark

Work with Spark and aggregate data from your application, like count the victories or find the highscore, or best player by what ever metric suits your game.

As a first step, read your data from a simple file.

Then try to connect Spark to Kafka and read it out as a stream.

Tutorial for Sparks and Kafka integration

Rittman Mead

[Spark Streaming and Kafka, Part 3 - Analysing Data in Scala and Spark]

In my first two blog posts of the Spark Streaming and Kafka series - Part 1 - Creating a New Kafka Connector and Part 2 - Configuring a Kafka Connector - I showed how to create a new custom Kafka Connector and how to set it up on a Kafka